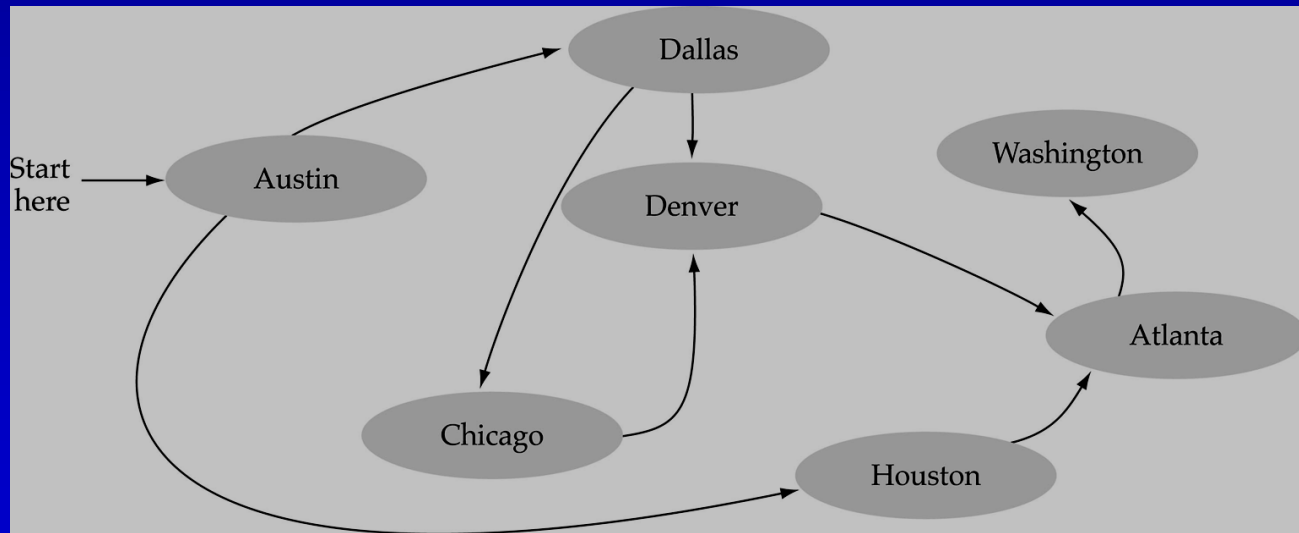


Graphs

What is a graph?

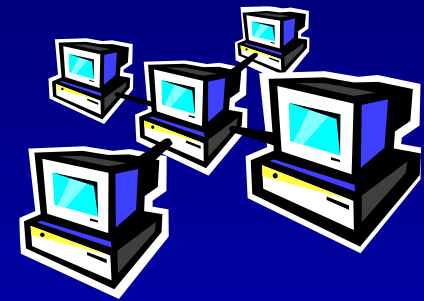
- A data structure that consists of a set of nodes (*vertices*) and a set of edges between the vertices.
- The set of edges describes relationships among the vertices.



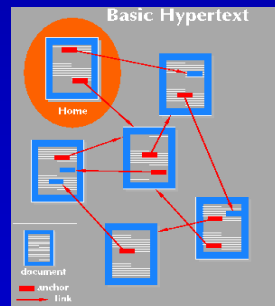
Applications



Maps



Computer networks



Hypertext



Circuits

Formal definition of graphs

- A graph G is defined as follows:

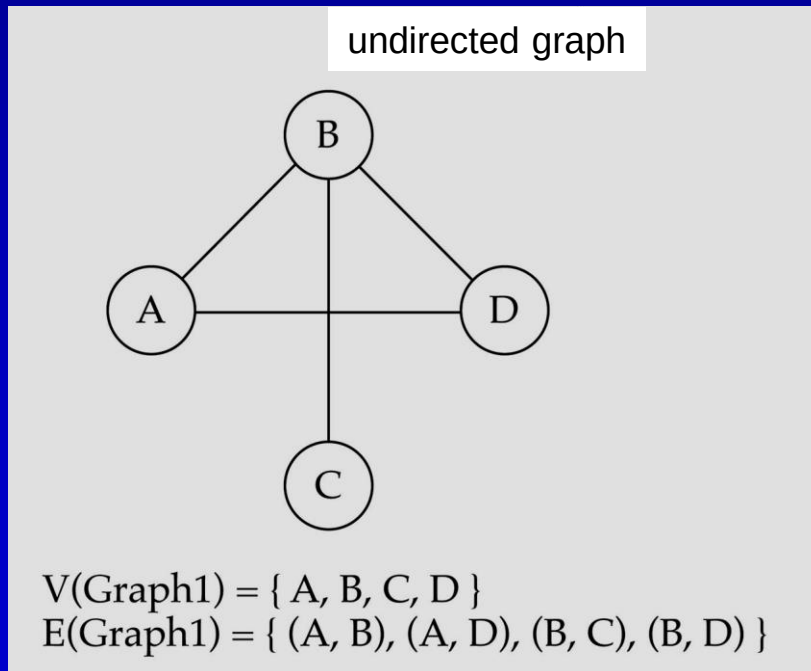
$$G=(V,E)$$

V : a finite, nonempty set of vertices

E : a set of edges (pairs of vertices)

Undirected graphs

- When the edges in a graph have no direction, the graph is called *undirected*

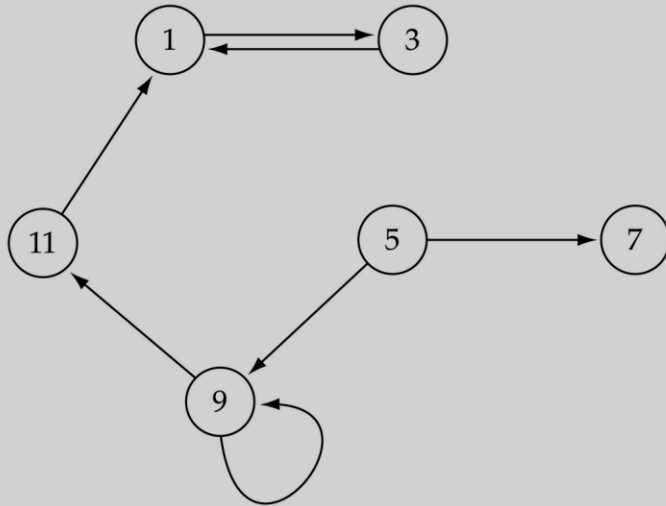


The **order** of vertices in E
is not important for
undirected graphs!!

Directed graphs

- When the edges in a graph have a direction, the graph is called *directed*.

(b) Graph2 is a directed graph.



$V(\text{Graph2}) = \{ 1, 3, 5, 7, 9, 11 \}$

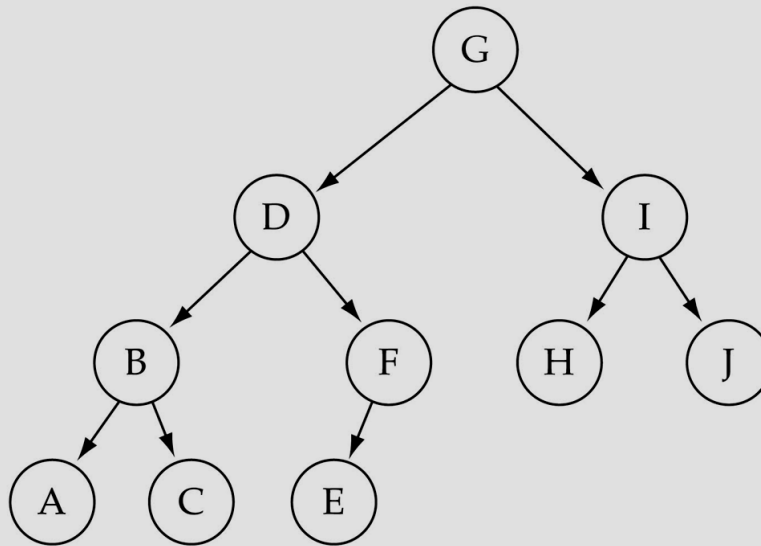
$E(\text{Graph2}) = \{(1,3) (3,1) (5,9) (9,11) (5,7), (9, 9), (11, 1) \}$

The **order** of vertices in E
is important for
directed graphs!!

Trees vs graphs

- Trees are special cases of graphs!!

(c) Graph3 is a directed graph.



$V(\text{Graph3}) = \{ A, B, C, D, E, F, G, H, I, J \}$

$E(\text{Graph3}) = \{ (G, D), (G, I), (D, B), (D, F), (I, H), (I, J), (B, A), (B, C), (F, E) \}$

Graph terminology

- Adjacent nodes: two nodes are adjacent if they are connected by an edge



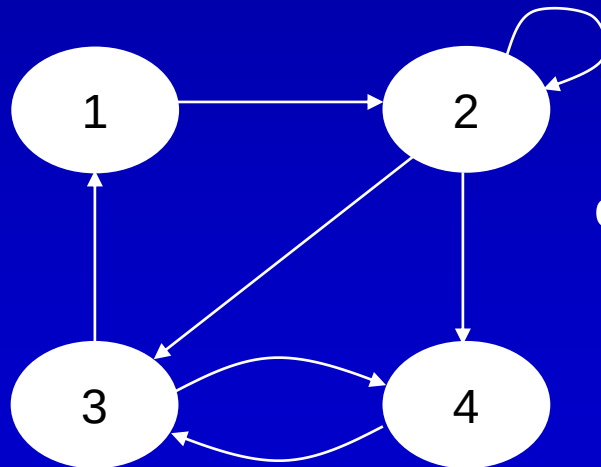
7 is adjacent **from** 5
or
5 is adjacent **to** 7



7 is adjacent **from/to** 5
or
5 is adjacent **from/to** 7

Graph terminology

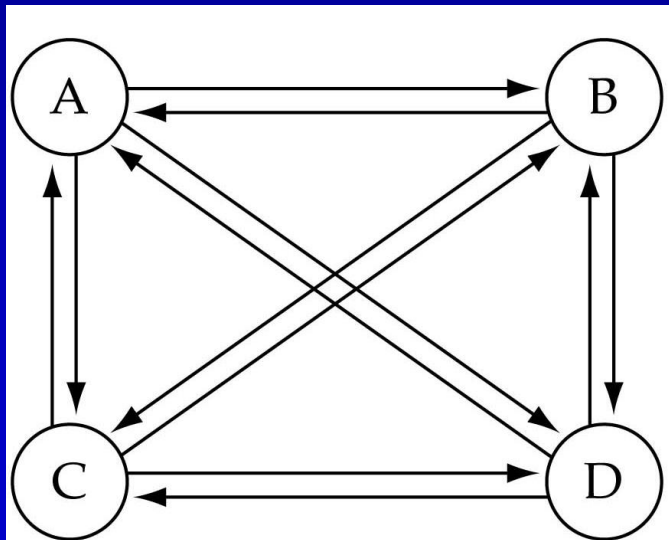
- Path: a sequence of vertices that connect two nodes in a graph.



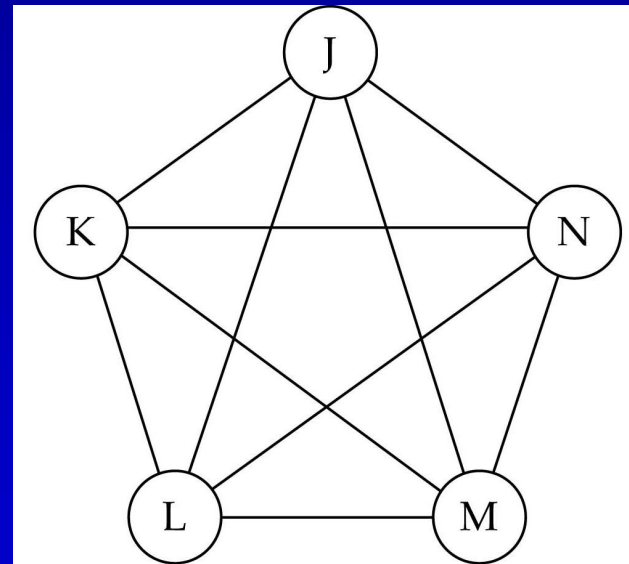
e.g., a path from 1 to 4:
<1, 2, 3, 4>

Graph terminology

- Complete graph: a graph in which every vertex is directly connected to every other vertex



(a) Complete directed graph.



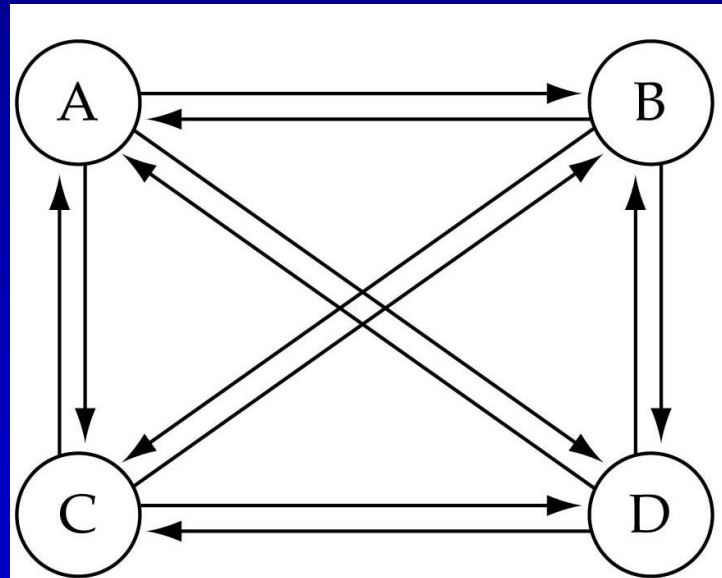
(b) Complete undirected graph.

Graph terminology (cont.)

- What is the number of edges E in a complete directed graph with V vertices?

$$E = V * (V - 1)$$

$$\text{or } E = O(V^2)$$



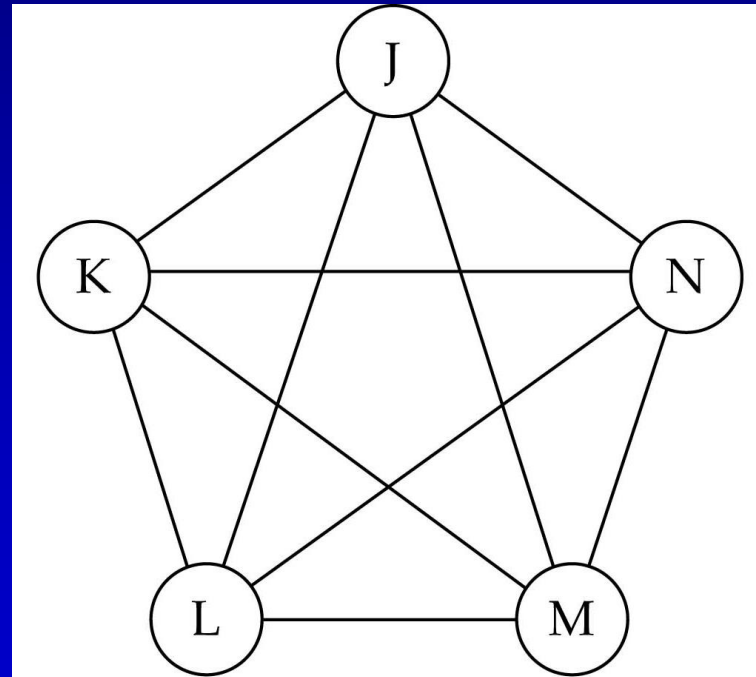
(a) Complete directed graph.

Graph terminology (cont.)

- What is the number of edges E in a complete undirected graph with V vertices?

$$E = V * (V - 1) / 2$$

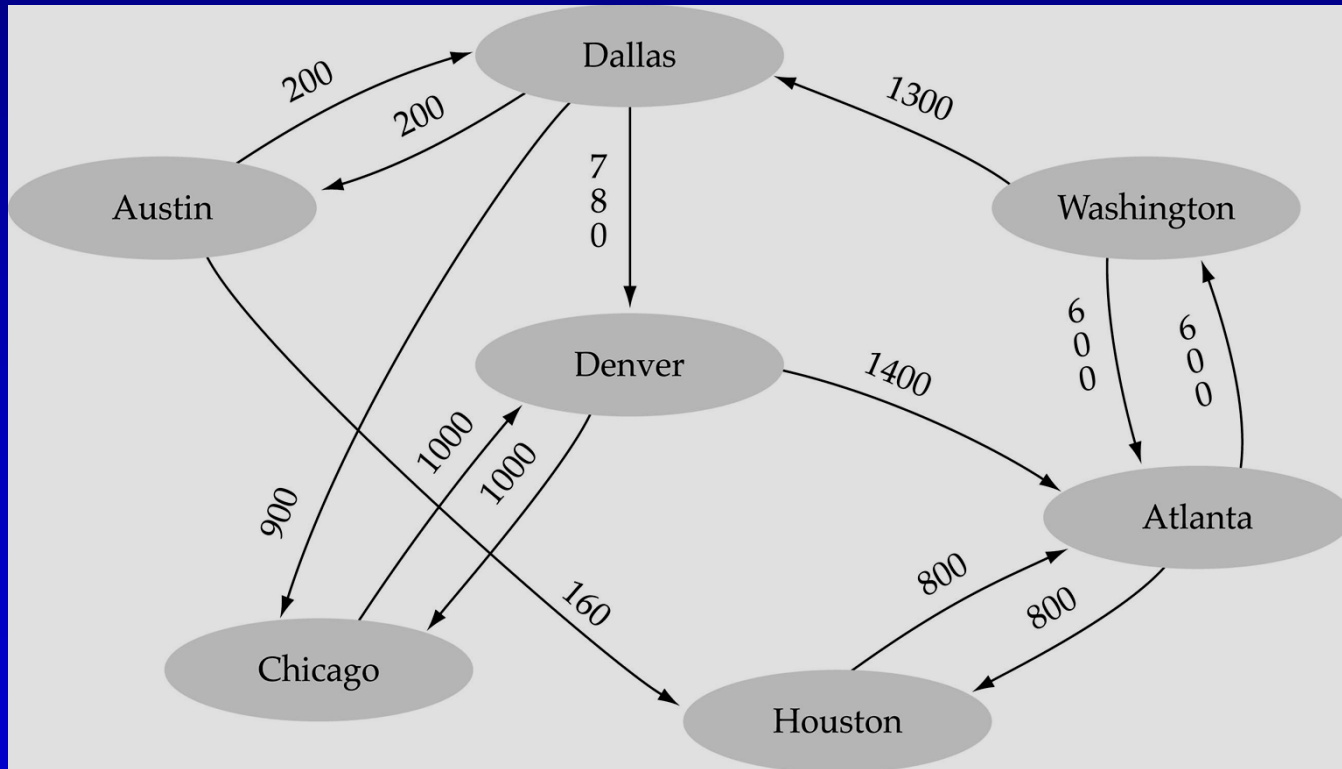
$$\text{or } E = O(V^2)$$



(b) Complete undirected graph.

Graph terminology (cont.)

- Weighted graph: a graph in which each edge carries a value

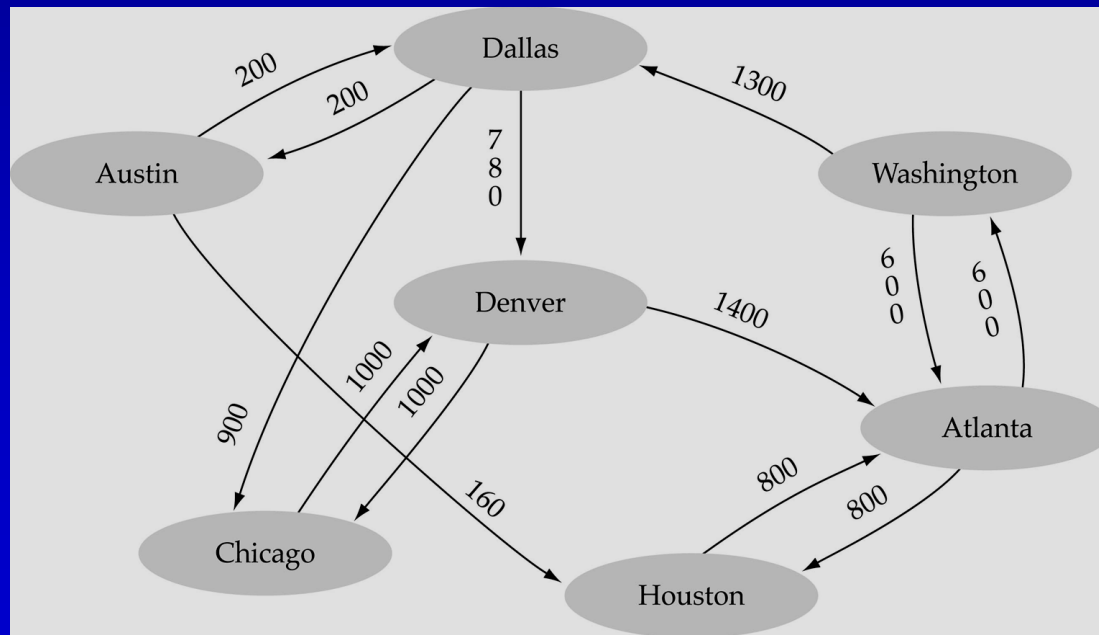


Graph Implementation

- Array-based
- Linked-list-based

Array-based implementation

- Use a 1D array to represent the vertices
- Use a 2D array (i.e., adjacency matrix) to represent the edges



Array-based implementation (cont'd)

graph

.numVertices 7

.vertices

[0]	"Atlanta"	"
[1]	"Austin"	"
[2]	"Chicago"	"
[3]	"Dallas"	"
[4]	"Denver"	"
[5]	"Houston"	"
[6]	"Washington"	"
[7]		
[8]		
[9]		

.edges

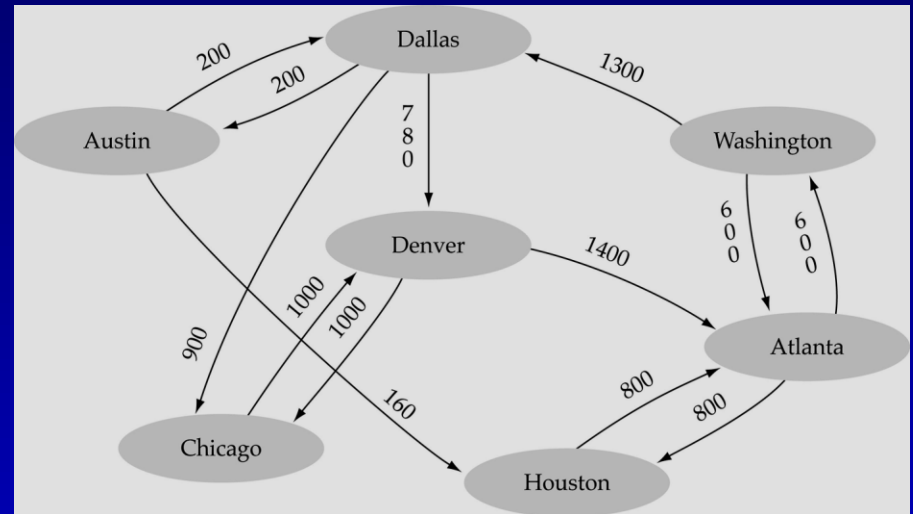
[0]	0	0	0	0	0	800	600	•	•	•
[1]	0	0	0	200	0	160	0	•	•	•
[2]	0	0	0	0	1000	0	0	•	•	•
[3]	0	200	900	0	780	0	0	•	•	•
[4]	1400	0	1000	0	0	0	0	•	•	•
[5]	800	0	0	0	0	0	0	•	•	•
[6]	600	0	0	1300	0	0	0	•	•	•
[7]	•	•	•	•	•	•	•	•	•	•
[8]	•	•	•	•	•	•	•	•	•	•
[9]	•	•	•	•	•	•	•	•	•	•

[0] [1] [2] [3] [4] [5] [6] [7] [8] [9]

(Array positions marked '•' are undefined)

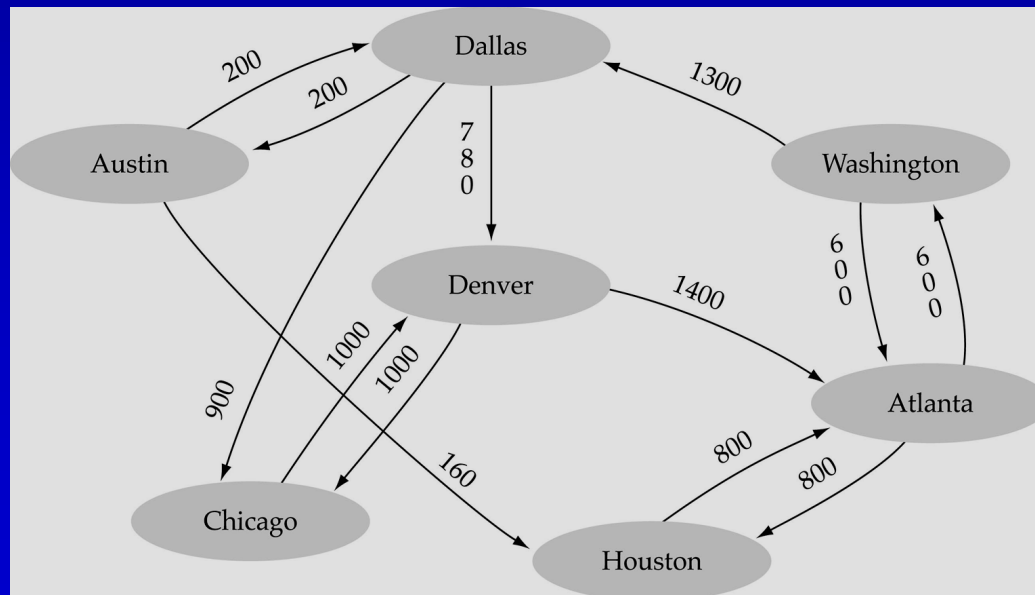
Array-Based Implementation (cont.)

- Memory required
 - $O(V+V^2)=O(V^2)$
- Preferred when
 - The graph is **dense**: $E = O(V^2)$
- Advantage
 - Can quickly determine if there is an edge between two vertices
- Disadvantage
 - No quick way to determine the vertices adjacent **from** another vertex

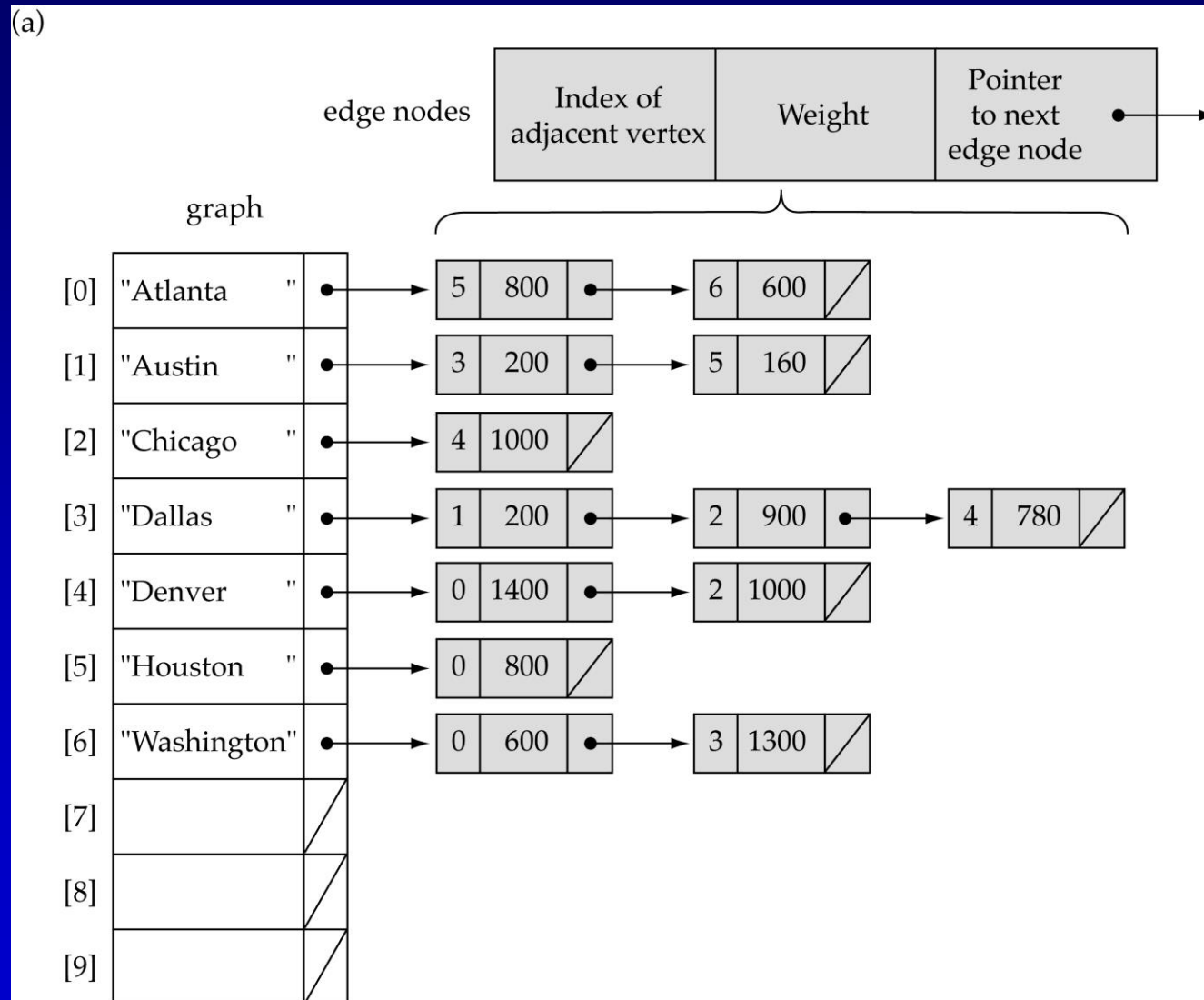


Linked-list-based implementation

- Use a 1D array to represent the vertices
- Use a list for each vertex v which contains the vertices which are adjacent from v (adjacency list)

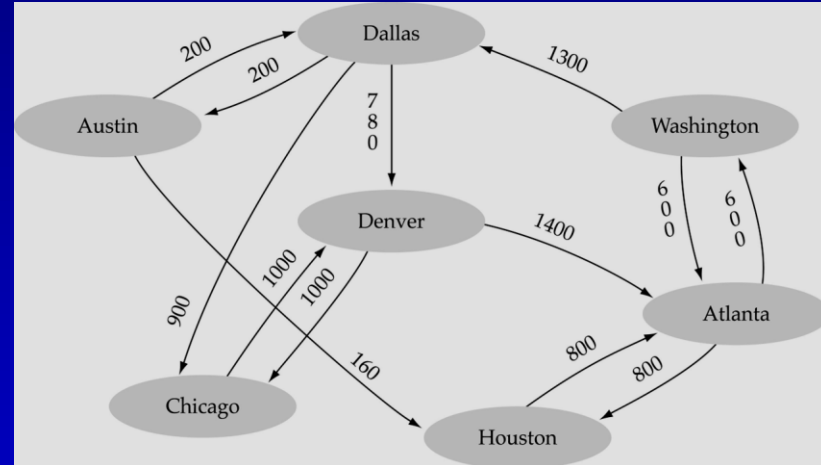


Linked-list-based implementation (cont'd)



Link-List-based Implementation (cont.)

- Memory required
 - $O(V + E)$
- Disadvantage
 - No quick way to determine whether there is an edge between vertices u and v
- Advantage
 - Can quickly determine the vertices adjacent **from** a given vertex



Graph specification based on adjacency matrix representation

```
const int NULL_EDGE = 0;

template<class VertexType>
class GraphType {
public:
    GraphType(int);
    ~GraphType();
    void MakeEmpty();
    bool IsEmpty() const;
    bool IsFull() const;
    void AddVertex(VertexType);
    void AddEdge(VertexType, VertexType, int);
    int WeightIs(VertexType, VertexType);
    void GetToVertices(VertexType, QueType<VertexType>&);
    void ClearMarks();
    void MarkVertex(VertexType);
    bool IsMarked(VertexType) const;

private:
    int numVertices;
    int maxVertices;
    VertexType* vertices;
    int **edges;
};
```

```
template<class VertexType>
GraphType<VertexType>::GraphType(int maxV)
{
    numVertices = 0;
    maxVertices = maxV;

    vertices = new VertexType[maxV];

    edges = new int[maxV];
    for(int i = 0; i < maxV; i++)
        edges[i] = new int[maxV];
}
```

```
template<class VertexType>
GraphType<VertexType>::~~GraphType()
{
    delete [] vertices;

    for(int i = 0; i < maxVertices; i++)
        delete [] edges[i];
    delete [] edges;

}
```

```

void GraphType<VertexType>::AddVertex(VertexType
    vertex)
{
    vertices[numVertices] = vertex;

    for(int index = 0; index < numVertices; index++) {
        edges[numVertices][index] = NULL_EDGE;
        edges[index][numVertices] = NULL_EDGE;
    }

    numVertices++;
}

```

graph

.numVertices 7												
.vertices		.edges										
[0]	"Atlanta"	[0]	0	0	0	0	0	800	600	•	•	•
[1]	"Austin"	[1]	0	0	0	200	0	160	0	•	•	•
[2]	"Chicago"	[2]	0	0	0	0	1000	0	0	•	•	•
[3]	"Dallas"	[3]	0	200	900	0	780	0	0	•	•	•
[4]	"Denver"	[4]	1400	0	1000	0	0	0	0	•	•	•
[5]	"Houston"	[5]	800	0	0	0	0	0	0	•	•	•
[6]	"Washington"	[6]	600	0	0	1300	0	0	0	•	•	•
[7]		[7]	•	•	•	•	•	•	•	•	•	•
[8]		[8]	•	•	•	•	•	•	•	•	•	•
[9]		[9]	•	•	•	•	•	•	•	•	•	•

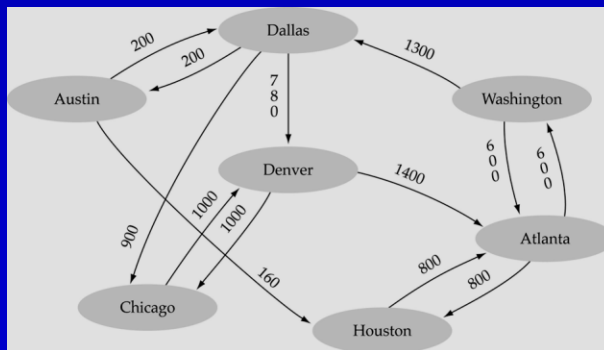
(Array positions marked '•' are undefined)

```

template<class VertexType>
void GraphType<VertexType>::AddEdge(VertexType
    fromVertex, VertexType toVertex, int weight)
{
    int row;
    int column;

    row = IndexIs(vertices, fromVertex);
    col = IndexIs(vertices, toVertex);
    edges[row][col] = weight;
}

```



graph

.numVertices 7		.vertices		.edges	
[0]	"Atlanta"			[0]	0 0 0 0 0 800 600 * * *
[1]	"Austin"			[1]	0 0 0 200 0 160 0 * * *
[2]	"Chicago"			[2]	0 0 0 0 1000 0 0 * * *
[3]	"Dallas"			[3]	0 200 900 0 780 0 0 * * *
[4]	"Denver"			[4]	1400 0 1000 0 0 0 0 * * *
[5]	"Houston"			[5]	800 0 0 0 0 0 0 * * *
[6]	"Washington"			[6]	600 0 0 1300 0 0 0 * * *
[7]				[7]	* * * * * * * * *
[8]				[8]	* * * * * * * * *
[9]				[9]	* * * * * * * * *

(Array positions marked '*' are undefined)